

$A \geq B$ (A-geq-B) - Claude Code Implementation Brief

A build-oriented specification for Claude Code and human review.

Purpose: Give Claude Code a precise, buildable specification for the OCaml inequality prover project.

Project name: $A \geq B$, pronounced "A-geq-B".

Repository name: `a-geq-b`.

Main language: OCaml.

Core rule: Certificate checker first, prover second, SDP/Coq last.

1. One-sentence project definition

$A \geq B$ is a standalone OCaml system that proves polynomial inequalities by converting $A \geq B$ into $A - B \geq 0$, searching for a sum-of-squares certificate, and verifying that certificate exactly using rational polynomial arithmetic.

A proof certificate has the form:

$A - B = \sum_i c_i * q_i(x)^2$
where $c_i \in \mathbb{Q}$, $c_i \geq 0$, and $q_i \in \mathbb{Q}[x_1, \dots, x_n]$.

If the identity checks exactly, the inequality is proven because squares are nonnegative and nonnegative rational multiples of squares remain nonnegative.

2. Non-negotiable implementation philosophy

Build the project in the LCF/certificate style:

1. The engine is untrusted. It may use heuristics, pattern search, numerical solvers, and Claude-generated code.
2. The checker is trusted. It only verifies certificates using exact arithmetic.
3. The final program must never say `PROVED` unless the checker accepts the certificate.
4. `NO_CERT_FOUND` is not the same as false. It means the current prover could not find a supported certificate.

Do not begin with the SDP solver or Coq extraction. First build a small exact checker that can validate hand-written certificates.

3. MVP scope

v1 supports

- Polynomial inequalities over the real numbers.
- Rational coefficients.
- Basic variables such as $a, b, c, x, y, z, x1$.
- Operators: $+$, $-$, $*$, $^$ with nonnegative integer exponents.
- Inequalities: $\geq$ and $\leq$.
- Sum-of-squares certificates with rational coefficients.
- Exact certificate checking.
- Human-readable proof output.
- LaTeX proof output.
- A CLI.

v1 attempts automatically

- Degree-2 quadratic-form inequalities.
- Simple square identities such as $(a-b)^2$.
- Pairwise-difference decompositions in variables a, b, c .
- Later: Gram matrix / SDP search.

v1 does not promise

- Arbitrary nonlinear inequality proving.
- Full real quantifier elimination.
- Full Coq/Lean integration.
- Arbitrary constraints such as $abc = 1$ or $a, b, c > 0$.
- Guaranteed success on all olympiad inequalities.

4. Required repository structure

Use this layout unless there is a strong reason to deviate:

```
a-geq-b/  
  dune-project  
  README.md  
  CLAUDE.md  
  bin/  
    main.ml  
  lib/  
    rational.ml  
    monomial.ml  
    polynomial.ml  
    ast.ml  
    parser.ml  
    normalize.ml  
    certificate.ml
```

```

checker.ml
prover.ml
pretty.ml
latex.ml
test/
  test_polynomial.ml
  test_parser.ml
  test_checker.ml
  test_prover.ml
examples/
  basic_three_var.json
  amgm2.json
  corrupted_certificate.json
docs/
  design.md
  proof_architecture.md

```

Use Dune. Use Alcotest or OUnit2 for tests. Use Zarith or another exact rational implementation.

5. Data model

5.1 Monomials

A monomial is an exponent vector.

For variables `[a,b,c]`, the monomial `a^2 b c^3` is represented by:

```

type monomial = int list
(* [2;1;3] means a^2 b^1 c^3 *)

```

Invariants:

- Exponents are nonnegative integers.
- The vector length equals the number of variables in the polynomial context.
- The variable ordering is explicit and stored with the claim/certificate.

5.2 Polynomials

A polynomial is a finite map from monomials to rational coefficients.

```

type poly = (monomial, Q.t) Map.t

```

Invariants:

- No zero coefficients are stored.
- Like terms are always combined.
- Polynomials are normalized before equality checks.
- Equality is structural equality of canonical normal forms.

5.3 Core polynomial operations

Implement and test:

- zero, one, var.
- add, neg, sub.
- mul, pow_nat.
- scalar_mul.
- normalize.
- equal.
- degree.
- variables / variable context utilities.
- to_string and to_latex.

Property tests to add:

- $p + 0 = p$.
- $p - p = 0$.
- $p * 1 = p$.
- $p * q = q * p$.
- $(p + q) + r = p + (q + r)$.
- $p * (q + r) = p*q + p*r$.

6. Parser

The parser should accept strings like:

prove $a^2 + b^2 + c^2 \geq a*b + b*c + c*a$

It should parse into an AST, then convert the AST into two polynomials A and B.

Supported syntax

```
numbers:    0, 1, -3, 1/2
variables:  a, b, c, x, y, z, x1
operators:  +, -, *, ^
relations:  >=, <=
command:    prove
```

Implicit multiplication such as ab is convenient but can be added later. In v1, explicit $a*b$ is acceptable. If implicit multiplication is implemented, test it heavily.

Parsing priority

1. Parentheses.
2. Powers.

3. Multiplication.
4. Addition/subtraction.
5. Inequality relation.

7. Certificate format

Start with JSON certificates.

Example:

```
{
  "claim": "a^2+b^2+c^2 >= a*b+b*c+c*a",
  "variables": ["a", "b", "c"],
  "sos": [
    {"coeff": "1/2", "poly": "a-b"},
    {"coeff": "1/2", "poly": "b-c"},
    {"coeff": "1/2", "poly": "c-a"}
  ]
}
```

The certificate means:

$$A - B = 1/2(a-b)^2 + 1/2(b-c)^2 + 1/2(c-a)^2.$$

8. Checker

The checker is the most important component.

Given a claim $A \geq B$ and an SOS certificate, it must:

1. Parse and normalize $A - B$.
2. Parse every certificate term c_i, q_i .
3. Check $c_i \geq 0$ exactly.
4. Compute $\sum_i c_i * q_i^2$ exactly.
5. Normalize both sides.
6. Return **PROVED** only if the two normalized polynomials are exactly equal.

Pseudocode:

```
let check_certificate claim cert =
  let p = normalize (lhs claim - rhs claim) in
  let rhs =
    cert.sos
    |> List.map (fun term ->
      assert (term.coeff >= 0);
      scalar_mul term.coeff (mul term.poly term.poly))
    |> sum
    |> normalize
  in
  equal p rhs
```

Corrupted certificate tests are mandatory.

The checker must reject:

- Negative coefficients.
- Wrong expansions.
- Missing terms.
- Extra terms.
- Malformed rational numbers.
- Variables not listed in the variable context.

9. Human-readable output

For a successful proof, the CLI should print something like:

Claim:

$$a^2 + b^2 + c^2 \geq a*b + b*c + c*a$$

Equivalent to proving:

$$a^2 + b^2 + c^2 - a*b - b*c - c*a \geq 0$$

Certificate:

$$\begin{aligned} & a^2 + b^2 + c^2 - a*b - b*c - c*a \\ &= \frac{1}{2}(a-b)^2 + \frac{1}{2}(b-c)^2 + \frac{1}{2}(c-a)^2 \end{aligned}$$

Reason:

Each term on the right is a square multiplied by a nonnegative rational.
Therefore the original inequality holds.

Status: PROVED

Also output LaTeX for the certificate.

10. CLI commands

Implement:

```
a-geq-b prove "a^2+b^2+c^2 >= a*b+b*c+c*a"
a-geq-b check examples/basic_three_var.json
a-geq-b --latex prove "a^2+b^2 >= 2*a*b"
```

Expected statuses:

```
PROVED
NO_CERT_FOUND
INVALID_INPUT
CHECK_FAILED
```

Important: `prove` may call the automatic prover. But if the automatic prover finds a proposed certificate, it must still send it through `checker.ml` before printing `PROVED`.

11. Automatic prover stages

Stage A: no automatic prover

First, implement `check` only. Use hand-written certificates.

Acceptance criteria:

- The checker accepts the classic three-variable certificate.
- It rejects a corrupted coefficient.
- It rejects a corrupted polynomial.
- It prints a readable proof.

Stage B: quadratic-form prover

For homogeneous degree-2 polynomials, build the symmetric matrix `A` such that:

$$p(x) = x^T A x.$$

If `A` is positive semidefinite and decomposable rationally, emit an SOS certificate.

Target examples:

$$\begin{aligned} a^2 + b^2 &\geq 2ab \\ x^2 + y^2 + z^2 &\geq xy + yz + zx \\ 2x^2 + 2y^2 &\geq (x+y)^2 \end{aligned}$$

Stage C: pairwise-difference search

For variables `a, b, c`, try certificates built from:

$$\begin{aligned} a-b \\ b-c \\ c-a \end{aligned}$$

and simple monomial multiples of these.

This helps with symmetric inequalities tight at `a=b=c`.

Stage D: Gram / SDP engine

Only after Stages A-C work, add the Gram matrix method.

Tasks:

1. Generate a monomial basis `z` up to degree `d`.
2. Represent $z^T Q z$ symbolically.

3. Match coefficients with p to get linear constraints on entries of Q .
4. Call an external SDP solver.
5. Round approximate output to rational data.
6. Verify the resulting certificate with the checker.

The checker remains the final authority.

12. Tests that must exist

Polynomial tests

- Addition combines like terms.
- Multiplication expands correctly.
- Powers work.
- Zero terms are removed.
- Equality ignores construction history and compares normal forms.

Parser tests

- $a^2 + b^2 \geq 2ab$.
- Parentheses: $(a - b)^2$.
- Rationals: $\frac{1}{2}a^2$.
- Invalid syntax is rejected cleanly.

Checker tests

- Correct certificate accepted.
- Negative coefficient rejected.
- Wrong certificate rejected.
- Certificate with variable mismatch rejected.

Prover tests

- Quadratic examples prove automatically.
- `NO_CERT_FOUND` is returned for unsupported examples, not `PROVED`.

13. Milestones for Claude Code

Milestone 1: exact polynomial core

Build `monomial.ml`, `polynomial.ml`, and tests.

Exit condition:

```
dune runtest
```

passes polynomial arithmetic tests.

Milestone 2: parser and normalizer

Build `ast.ml`, `parser.ml`, `normalize.ml`.

Exit condition:

The system can parse:

```
prove a^2+b^2+c^2 >= a*b+b*c+c*a
```

and construct the polynomial:

```
a^2 + b^2 + c^2 - a*b - b*c - c*a
```

Milestone 3: certificate checker

Build `certificate.ml` and `checker.ml`.

Exit condition:

The checker accepts `examples/basic_three_var.json` and rejects `examples/corrupted_certificate.json`.

Milestone 4: proof printer

Build `pretty.ml` and `latex.ml`.

Exit condition:

CLI prints human-readable proof and LaTeX certificate.

Milestone 5: simple automatic prover

Build quadratic-form prover and simple square-pattern prover.

Exit condition:

At least 10 textbook inequalities prove automatically.

Milestone 6: optional frontend

Build a minimal local web UI or static UI that calls the CLI/backend.

Exit condition:

User can enter an inequality and see rendered proof output.

Milestone 7: optional SDP / Coq extensions

Add only after the core project works.

14. Do not do these too early

Do not begin with:

- Coq extraction.
- Full SDP integration.
- Full Putinar Positivstellensatz.
- Hilbert 17 rational-function certificates.
- Large olympiad corpus.
- A complicated frontend.

These are stretch goals, not foundations.

15. README pitch

Use this README pitch:

$A \geq B$ is a standalone OCaml prover for polynomial inequalities. Given a claim $A \geq B$, it searches for a sum-of-squares certificate for $A - B$ and verifies the certificate using exact rational arithmetic. The result is both machine-checkable and human-readable: the system prints an explicit algebraic identity showing why the inequality is true.

Example:

$$a^2 + b^2 + c^2 \geq a*b + b*c + c*a$$

becomes:

$$\begin{aligned} & a^2 + b^2 + c^2 - a*b - b*c - c*a \\ &= \frac{1}{2}(a-b)^2 + \frac{1}{2}(b-c)^2 + \frac{1}{2}(c-a)^2. \end{aligned}$$

Since the right-hand side is a sum of nonnegative squares, the inequality is proven.

16. Resume framing

Do not overclaim. A good bullet after v1 is:

Built $A \geq B$, an OCaml inequality prover that parses polynomial inequalities, searches for sum-of-squares certificates, and verifies proofs using exact rational arithmetic, producing human-readable algebraic certificates.

If the LCF architecture is clear:

Designed an LCF-style architecture separating an untrusted SOS search engine from a small exact checker, ensuring the system prints PROVED only after validating a machine-checkable certificate.

If the frontend exists:

Built a lightweight frontend that renders automatically generated sum-of-squares certificates as readable LaTeX proofs.

If SDP/Coq exists later:

Extended $A \geq B$ with Gram-matrix SOS search via semidefinite programming and an optional Coq-extracted certificate checker.